Architecting for performance
A top-down approach

Ionuţ Baloşin

Software Architect

www.ionutbalosin.com
@ionutbalosin

I T.A.K.E. Unconference | 2018

About Me

Ionuţ Baloşin

Software Architect

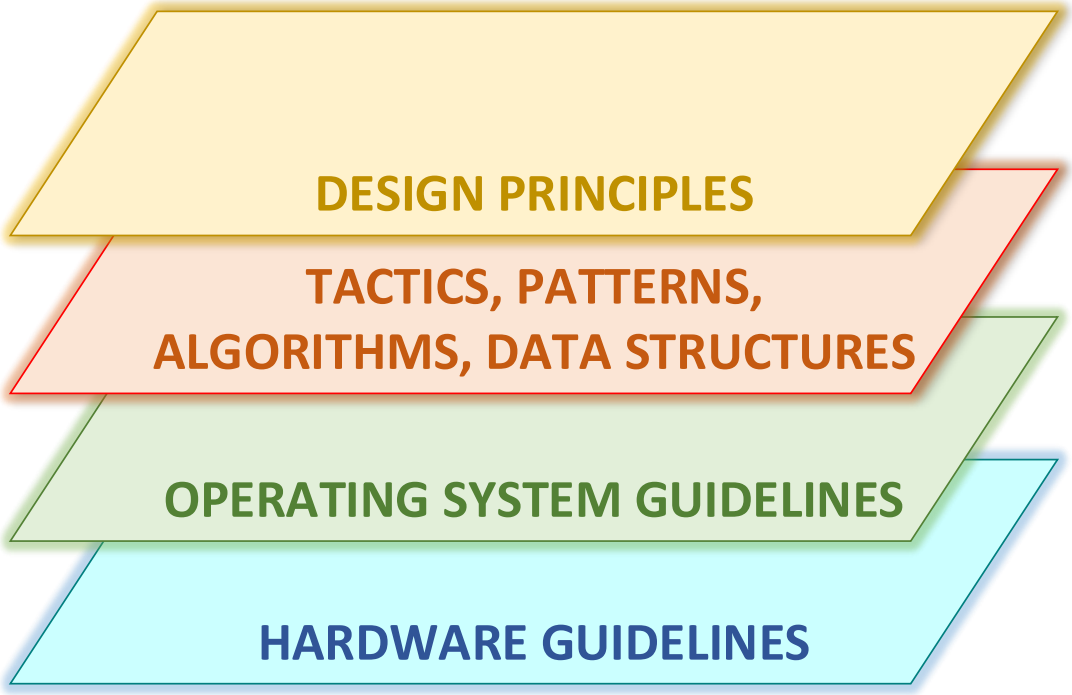
Technical Trainer

- Java Performance and Tuning
- Software Architecture

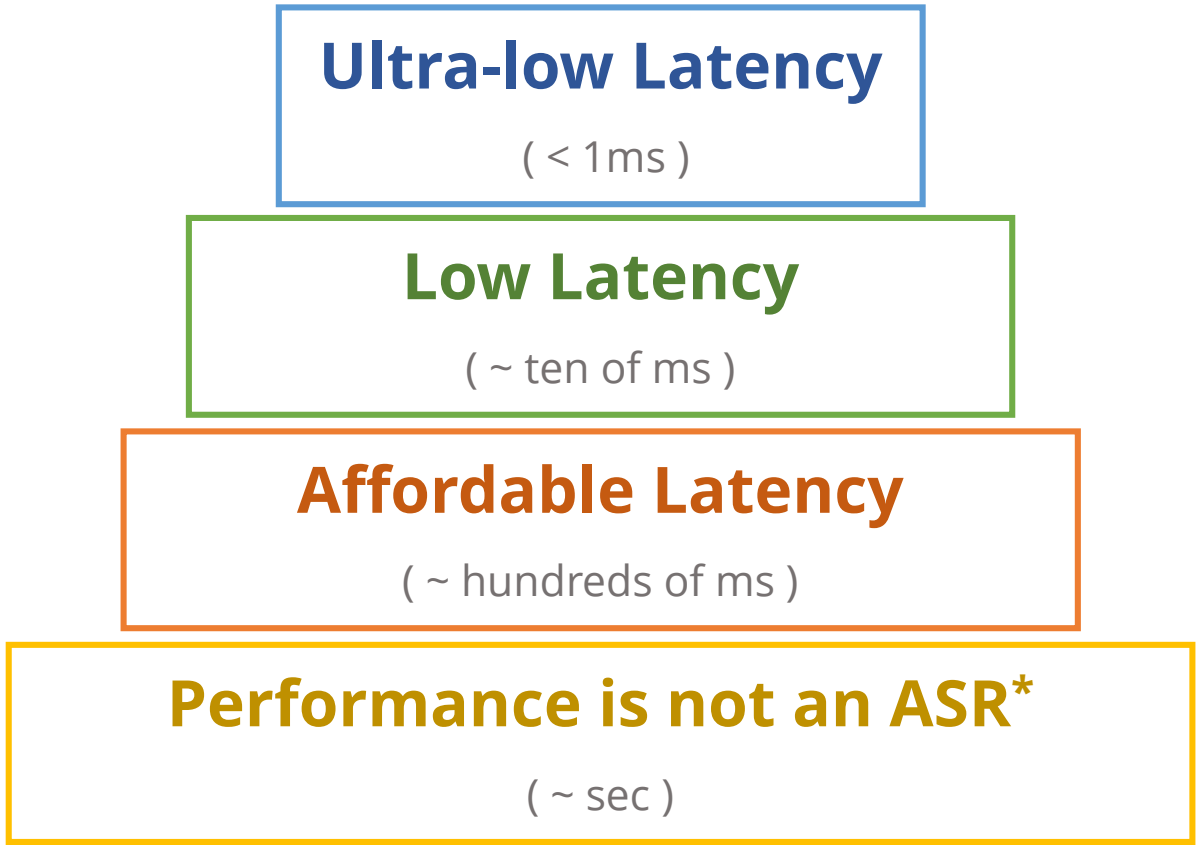
www.ionutbalosin.com

@ionutbalosin

Agenda


DESIGN PRINCIPLES
TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

My Latency Hierarchical Model



ASR – *Architecturally Significant Requirement*

What is Performance?

*"**Performance** it's **about time** and the software system's **ability to meet timing requirements**."*

"Software Architecture in Practice" - **Rick Kazman, Paul Clements, Len Bass**

[**Source:** https://www.infoq.com/articles/IT-industry-better-namings]

InfoQ

Does IT Industry Need Better Namings?

Like | Posted by **Ionut Balosin** on Apr 24, 2017. Estimated reading time: 13 minutes

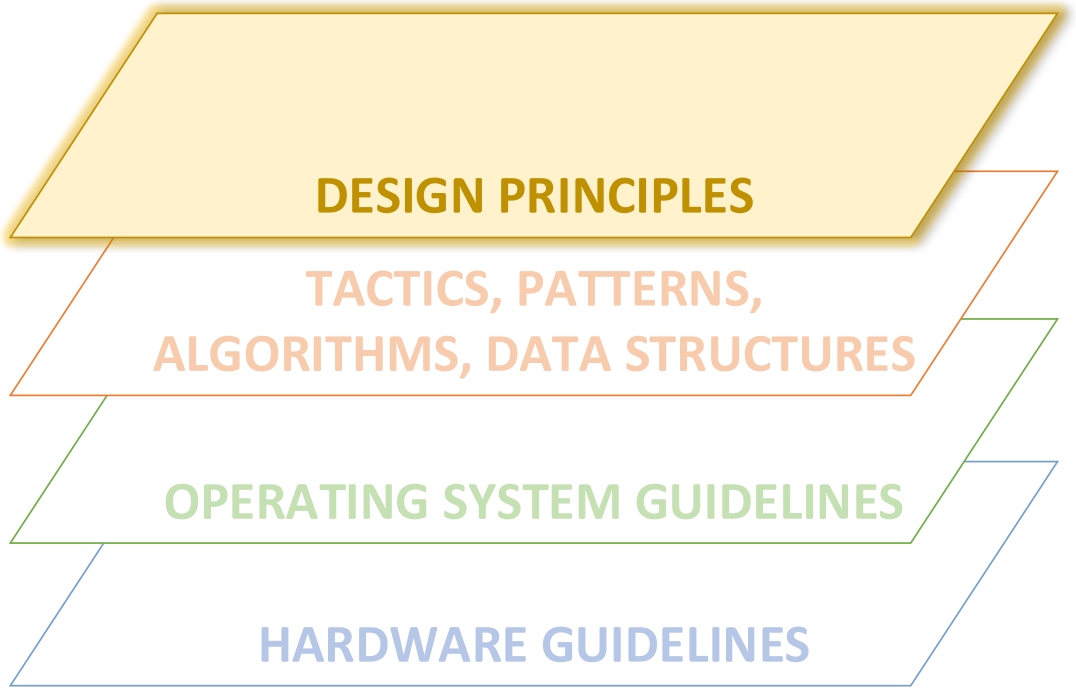
Non-functional requirements or quality attributes

Oftentimes, business requirements are drilled down in functional and non-functional specifications. In this context, the non-functional requirements refer to quality attributes such as performance, availability, scalability, security, usability, etc.

But let's try to better understand what's behind the non-functional terminology guided by two approaches: the first one is to search the word in the dictionary and the second one is to show the technical implications with regards to their achievability.

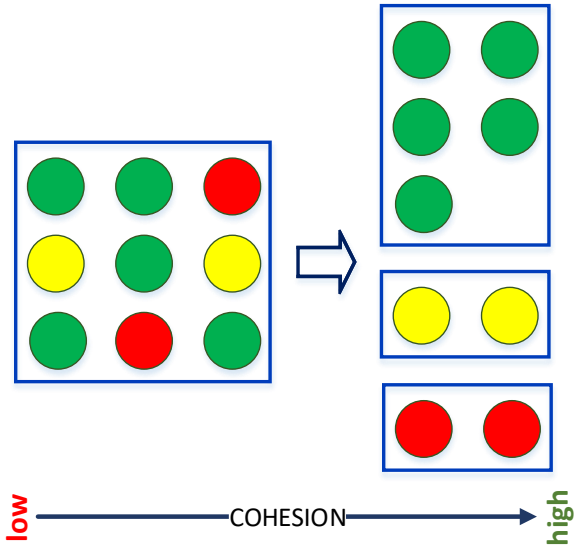
According to the dictionary, the *"non"* word means "*not, absence of, unimportant, worthless*". Based on this definition, the question which arises is: how can we name something non-functional and still having an impact on the architecture, since by semantics it is "worthless"? Why shouldn't we drop or put aside these requirements? Hence, the first inconsistency.

Following with the second approach, all non-functional requirements (e.g. security, usability) are implemented through functions (e.g. security via encryption, usability via a wizard), so they rely on


DESIGN PRINCIPLES
TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

Cohesion

Cohesion represents the degree to which the elements inside a module work / belong together.

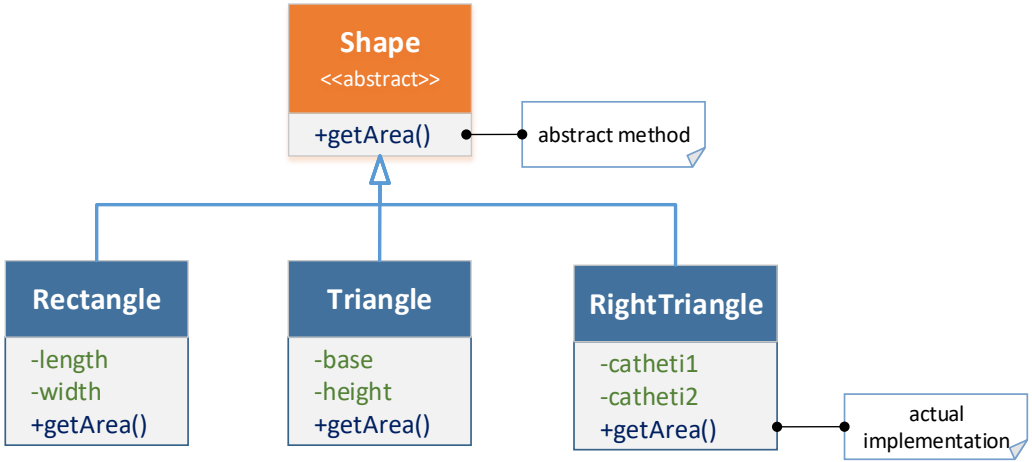


Cohesion => better locality =>
CPU iCache / dCache friendly

Classes must be cohesive, groups of class working together should be cohesive; however elements that are not related should be decoupled!

Abstractions

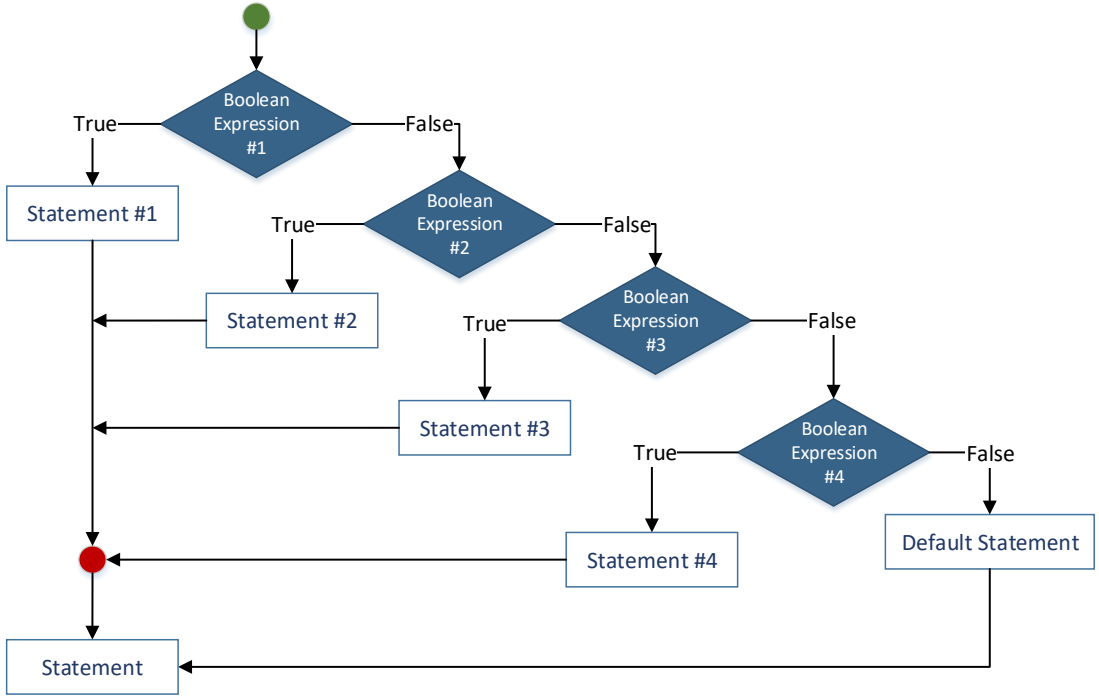
"The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise" - **Edsger Dijkstra**



Abstractions => polymorphism (e.g. virtual calls) => increased runtime cost

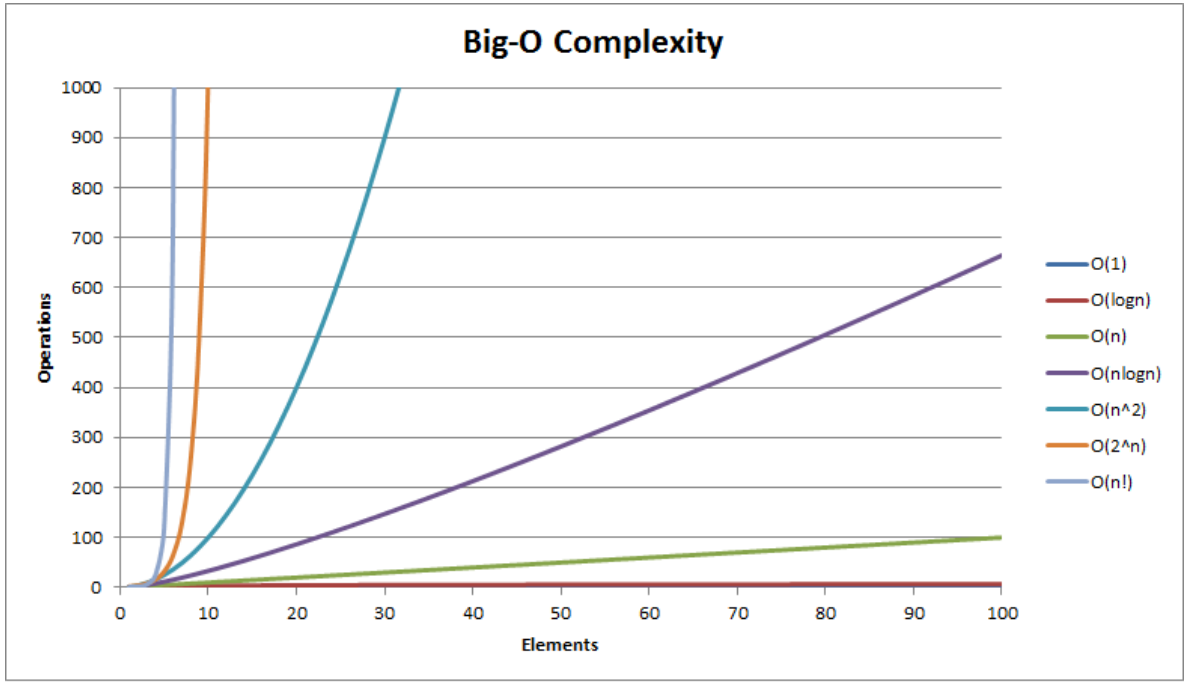
Cyclomatic Complexity

Cyclomatic complexity is the number of linearly independent paths through a program's source code.



Higher cyclomatic complexity => branch miss predictions => pipeline stalls

Algorithms Complexity



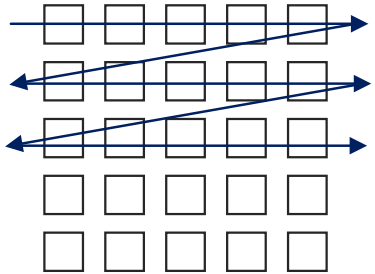
[**Source**: https://stackoverflow.com/questions/29927439/]

Service Time is a measure of algorithm complexity

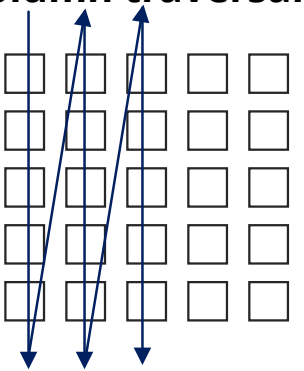
But … is it all about Big-O Complexity?

Matrix Traversal

Row traversal



Column traversal



Matrix Traversal

Row traversal


```
public long rowTraversal() {
    long sum = 0;
    for (int i = 0; i < mSize; i++)
        for (int j = 0; j < mSize; j++) {
            sum += matrix[i][j];
        }

    return sum;
}
```

Column traversal


```
public long columnTraversal() {
    long sum = 0;
    for (int i = 0; i < mSize; i++)
        for (int j = 0; j < mSize; j++) {
            sum += matrix[j][i];
        }

    return sum;
}
```

Matrix Traversal

Row traversal


```
public long rowTraversal() {
    long sum = 0;
    for (i
        fo
            i][j];
        }

    return sum;
}
```

$O(N^2)$

Column traversal


```
public long columnTraversal() {
    long sum = 0;
    for (
        f
            x[j][i];
        }

    return sum;
}
```

$O(N^2)$

Matrix Traversal

Matrix size	Row Traversal (ij) (ops/μs)	Column Traversal (ji) (ops/μs)
64 x 64	0.773	0.409
512 x 512	0.012	0.003
1024 x 1024	0.003	0.001
4096 x 4096	10^{-4}	10^{-5}
	$O(N^2)$	$O(N^2)$

higher is better

Matrix Traversal

Matrix size	Row Traversal (ij) (ops/μs)	Column Traversal (ji) (ops/μs)
64 x 64	0.773	0.409
512 x 512	0.012	0.003
1024 x 1024	0.003	0.001
4096 x 4096	10^{-4}	10^{-5}
	$O(N^2)$	$O(N^2)$

higher is better

Matrix Traversal

Matrix size	Row Traversal (ij) (ops/μs)	Column Traversal (ji) (ops/μs)
64 x 64	0.773	0.409
512 x 512	0.012	0.003
1024 x 1024	0.003	0.001
4096 x 4096	**10^{-4}**	**10^{-5}**
	$O(N^2)$	$O(N^2)$

higher is better

Why such noticeable difference ?

~ 1 order of magnitude

Matrix Traversal

Matrix size (4096 x 4096)	Row Traversal (ij)	Column Traversal (ji)
cycles per instruction	0.849	1.141
L1-dcache-loads	10^9 x 0.056	10^9 x 9.400
L1-dcache-load-misses	10^9 x 0.019	10^9 x 6
LLC-loads	10^9 x 0.014	10^9 x 6.100
LLC-load-misses	10^9 x 0.004	10^9 x 0.084
dTLB-loads	10^9 x 0.026	10^9 x 9.400
dTLB-load-misses	10^3 x 13	10^3 x 101

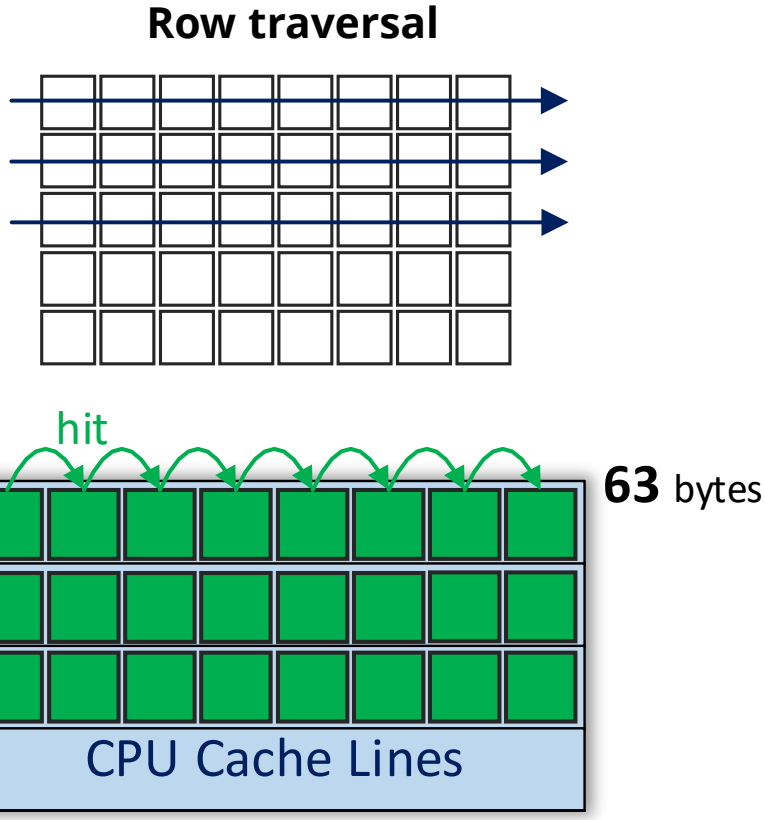
lower is better

Matrix Traversal

Matrix size (4096 x 4096)	Row Traversal (ij)	Column Traversal (ji)
cycles per instruction	0.849	1.141
L1-dcache-loads	10^9 x 0.056	10^9 x 9.400
L1-dcache-load-misses	10^9 x 0.019	10^9 x 6
LLC-loads	10^9 x 0.014	10^9 x 6.100
LLC-load-misses	10^9 x 0.004	10^9 x 0.084
dTLB-loads	10^9 x 0.026	10^9 x 9.400
dTLB-load-misses	10^3 x 13	10^3 x 101

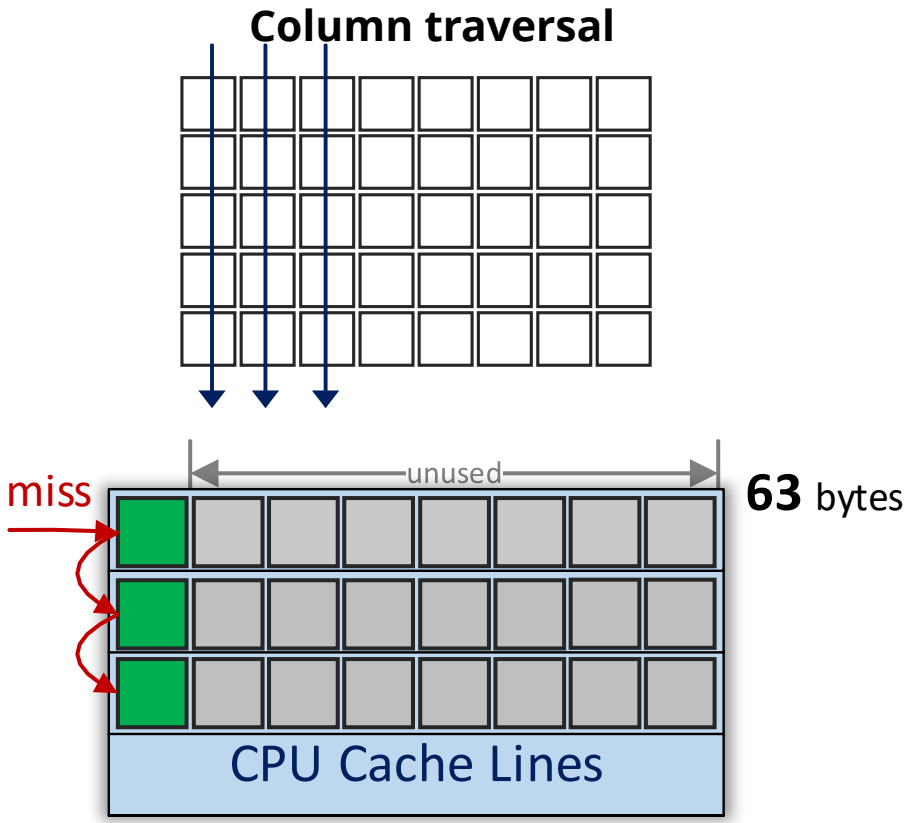
lower is better

Matrix Traversal



NB: Simplistic representation

Matrix Traversal



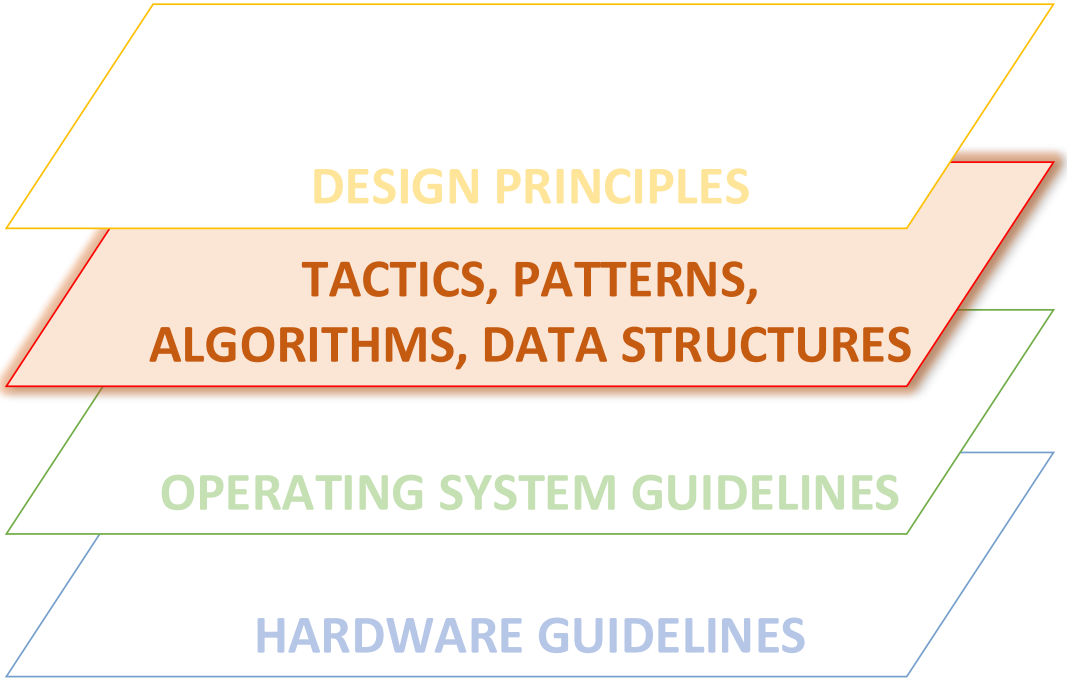
NB: Simplistic representation

On modern architectures Service Time is highly impacted by CPU caches

Big-O Complexity might win for huge data sets where CPU caches could not help

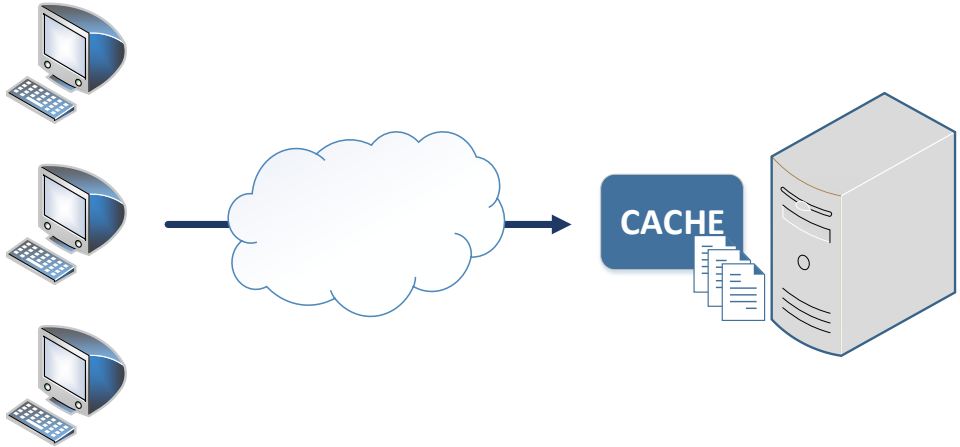
Recommendation

- reduce the code footprint as possible (e.g. small and clean methods)
- minimize object indirections as possible (e.g. array of primitives vs. array of objects)


DESIGN PRINCIPLES
TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

Caching

Caching stores application data in an optimized location to facilitate faster and easier retrieval



Data Patterns (e.g. read/write through, write behind, read ahead)

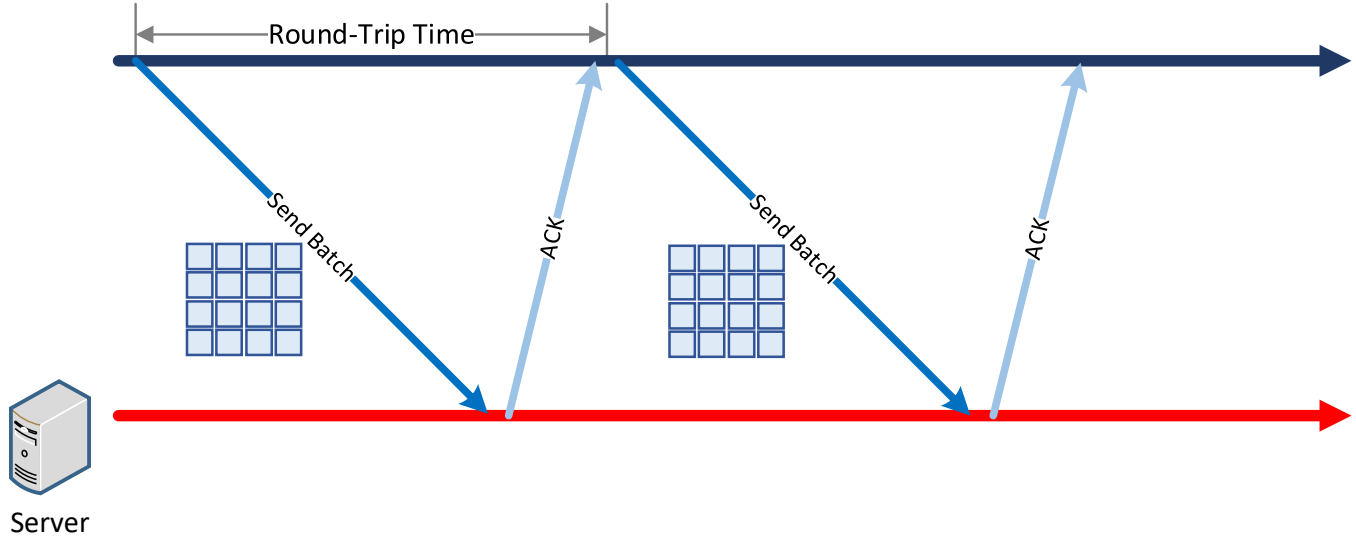
Eviction Algorithm (e.g. LRU, LFU, FIFO)

Fetching Strategy (e.g. pre-fetch, on-demand, predictive)

Topology (e.g. local, partitioned/distributed, partitioned-replicated)

Batching

Batching minimizes the number of server round trips, especially when data transfer is long.



Solution is limited by **bandwidth** and **Receiver's handling rate**

What is size(batch) for an optimal transfer (i.e. **max Bandwidth**, **min RTT**) ?

BBR Congestion Control

Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, Van Jacobson

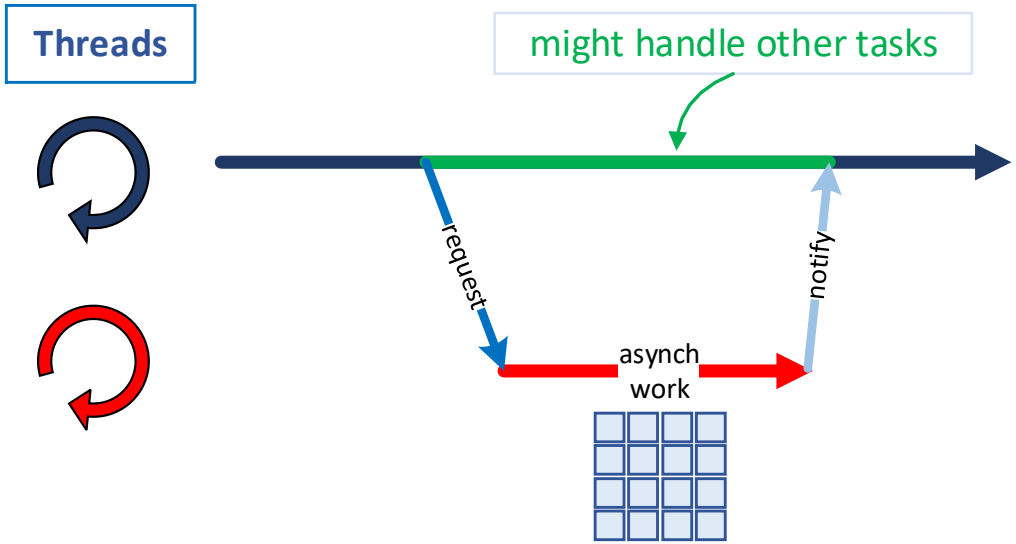
Bottleneck **B**andwidth and **R**ound-trip propagation time

walk toward (**max BW**, **min RTT**) point

[BBR Paper https://queue.acm.org/detail.cfm?id=3022184]

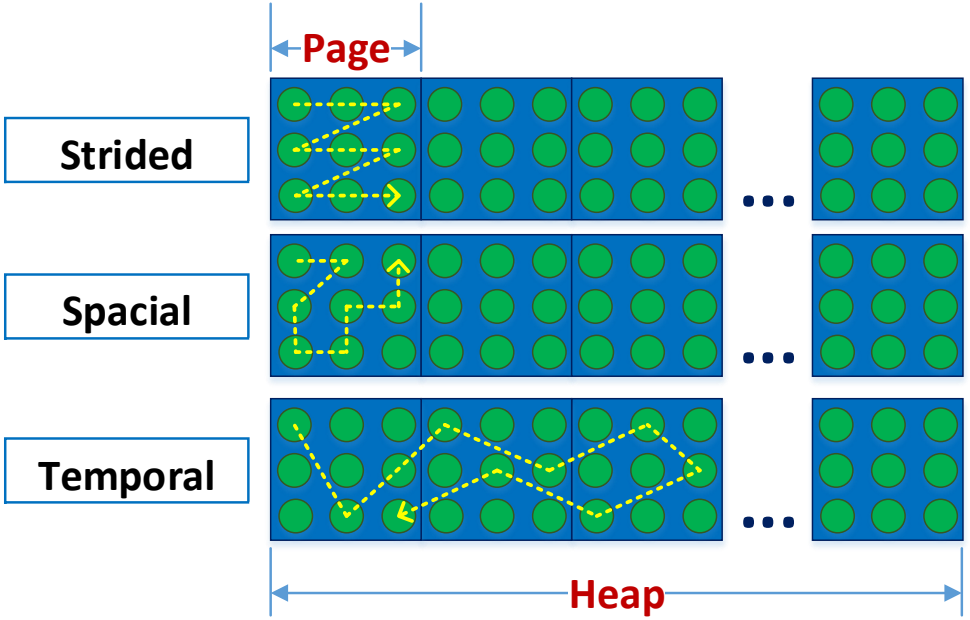
Design Asynchronous

"Design asynchronous by default, make it synchronous when it is needed" - **Martin Thompson**



Designing asynchronous and stateless is a good recipe for performance !

Memory Access Patterns



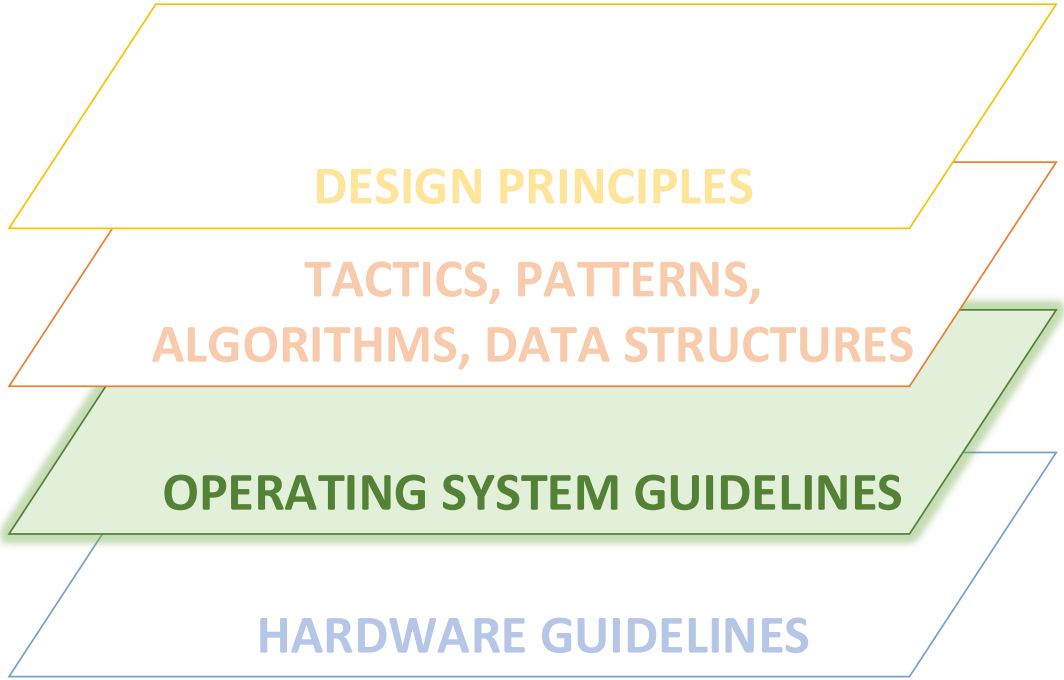
Strided - memory access is likely to follow a predictable pattern
Spatial - nearby memory is likely to be required soon
Temporal - memory accessed recently will likely be required again soon

Memory Access Patterns

Test scenario: traverse the memory in strided, spatial and temporal fashion by accessing elements from a long[] array of length 2GB / sizeof(long) (i.e. 2GB / 8) within 4GB of heap memory

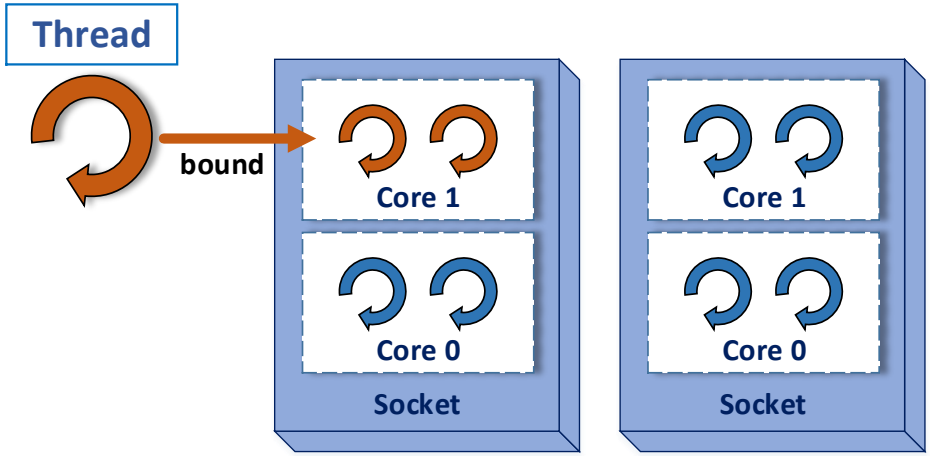
Access Pattern	Response Time (ns / op)
Strided	0.97
Spatial	4.40
Temporal	37.34

CPU: Intel i7-6700HQ Skylake
OS: Ubuntu 16.04.2


DESIGN PRINCIPLES
TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

Thread Affinity

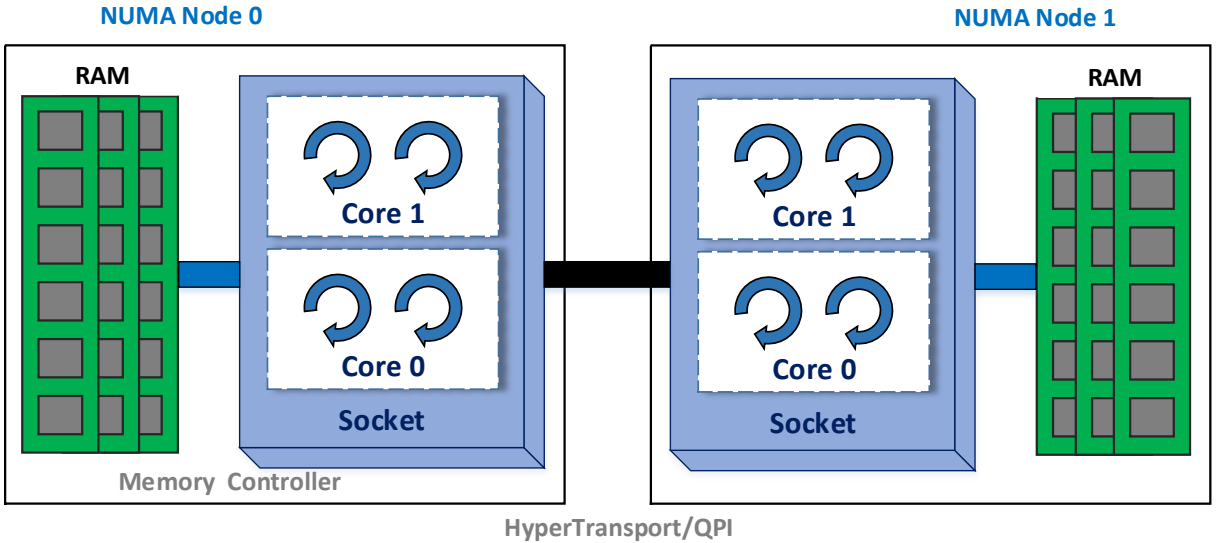
Thread Affinity binds a thread to a CPU or a range of CPUs so that the thread will execute only on the designated CPU or CPUs rather than any CPU



Thread affinity takes advantages on CPU cache memory.
When a thread migrates from one processor to another all cache lines have to be moved.

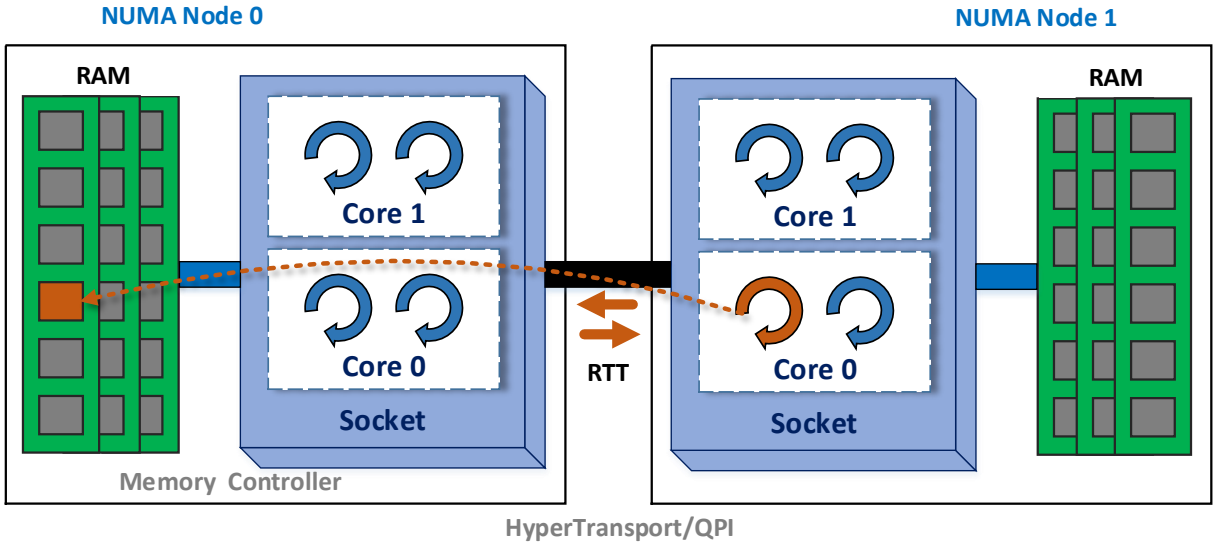
NUMA

Non-Uniform-Memory-Access (NUMA) is a memory design where the memory access time depends on the memory location relative to the processor



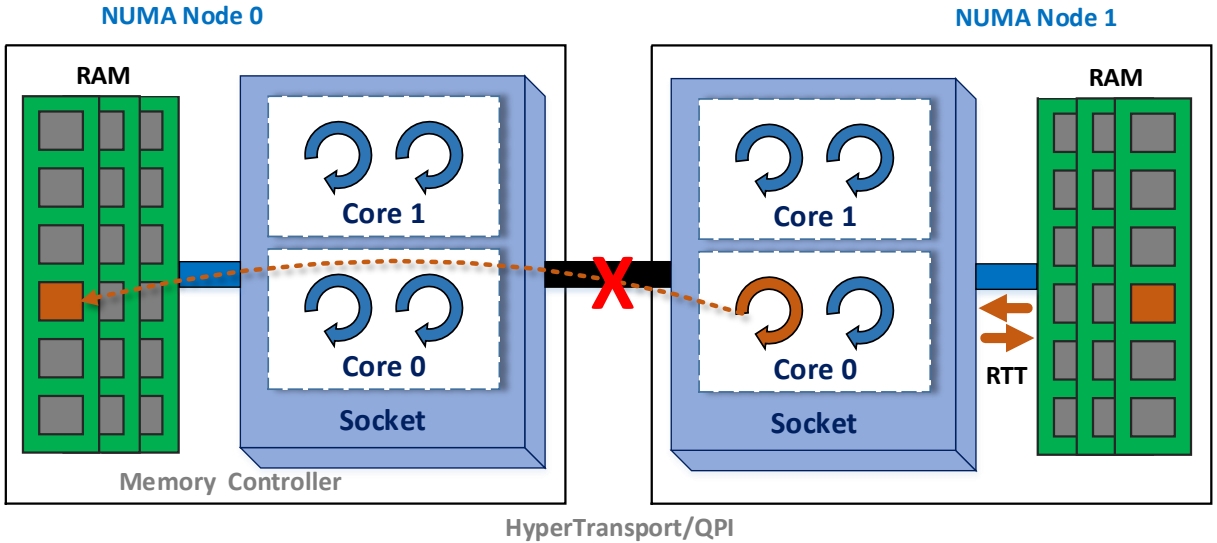
NUMA

Non-Uniform-Memory-Access (NUMA) is a memory design where the memory access time depends on the memory location relative to the processor



NUMA

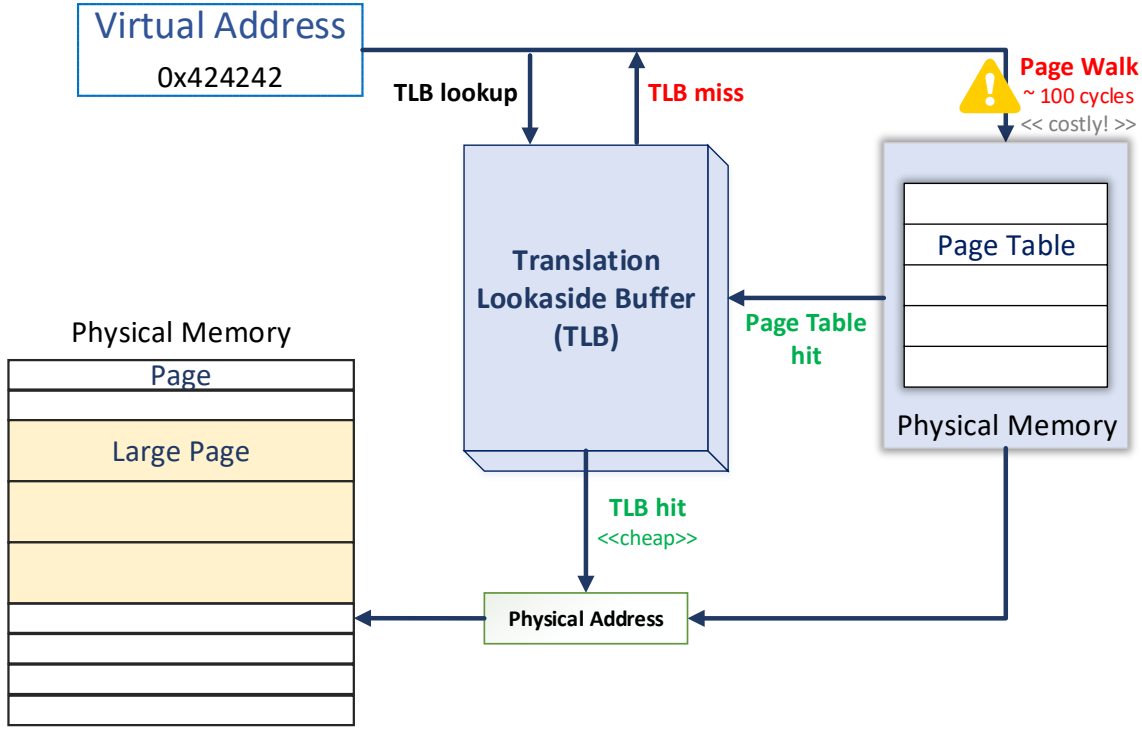
Non-Uniform-Memory-Access (NUMA) is a memory design where the memory access time depends on the memory location relative to the processor



JVM NUMA-aware allocator has been implemented to take advantage of local memory

Large Pages

Using Large Pages the TLB can represent larger memory rage hence reduces TLB misses and the number of page walks



Large Pages

Guidelines

- suitable for **intensive memory** applications with **large contiguous memory accesses**

Enable Large Pages when number of TLB misses and TLB Page walk take a significant amount of time (i.e. dtlb_load_misses_* CPU counters)

Large Pages

Not Recommended for ...

- short lived applications with **small working set**
- applications with **large** but **sparsely used heap**

RamFS & TmpFS

RamFS & TmpFS allocate a part of the physical memory to be used as a partition (e.g. write/read files).

Useful for applications which performs a lot disk reads/writes (e.g. logging, auditing)

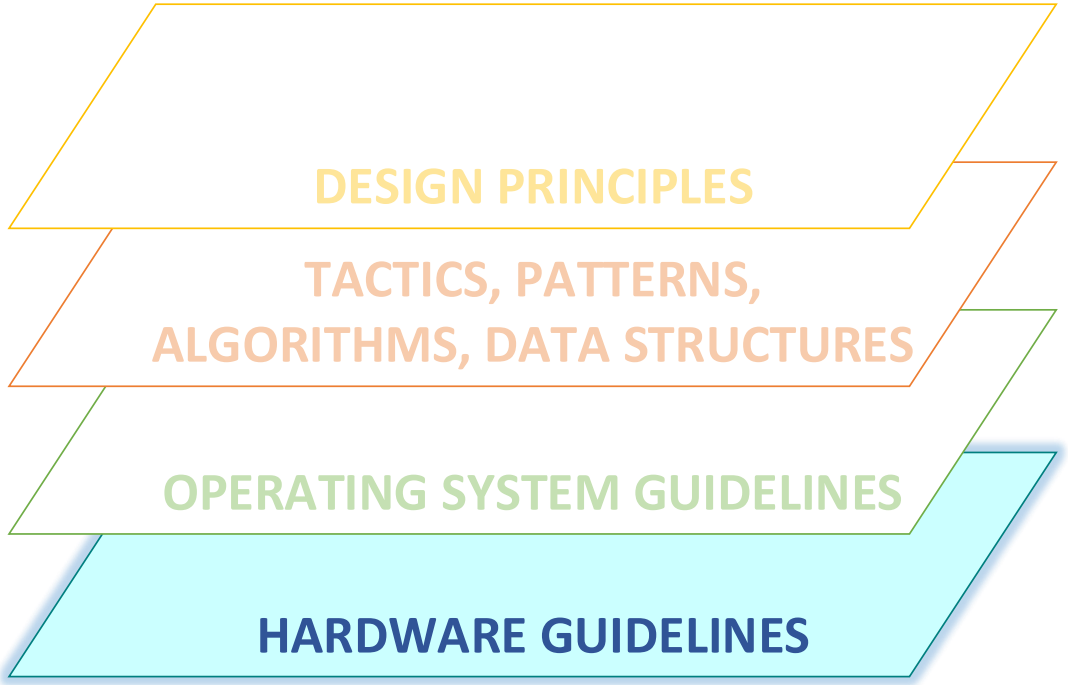
RamFS & TmpFS

Test scenario: sequentially reading/writing 8GB in chunks of 4KB/512 KB on HDD/SSD/RAMFS

	HDD (5,400RPM)		**SSD**		**RAMFS**	
Chunk	Read MB/s	Write MB/s	Read MB/s	Write MB/s	Read MB/s	Write MB/s
4K	128	99	964	742	7,971	4,420
512K	147	113	1,021	788	10,760	6,045

higher is better

NB: higher read rates are caused by buffers/caches effect


DESIGN PRINCIPLES
TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

False Sharing

False Sharing is purely a CPU Cache issue

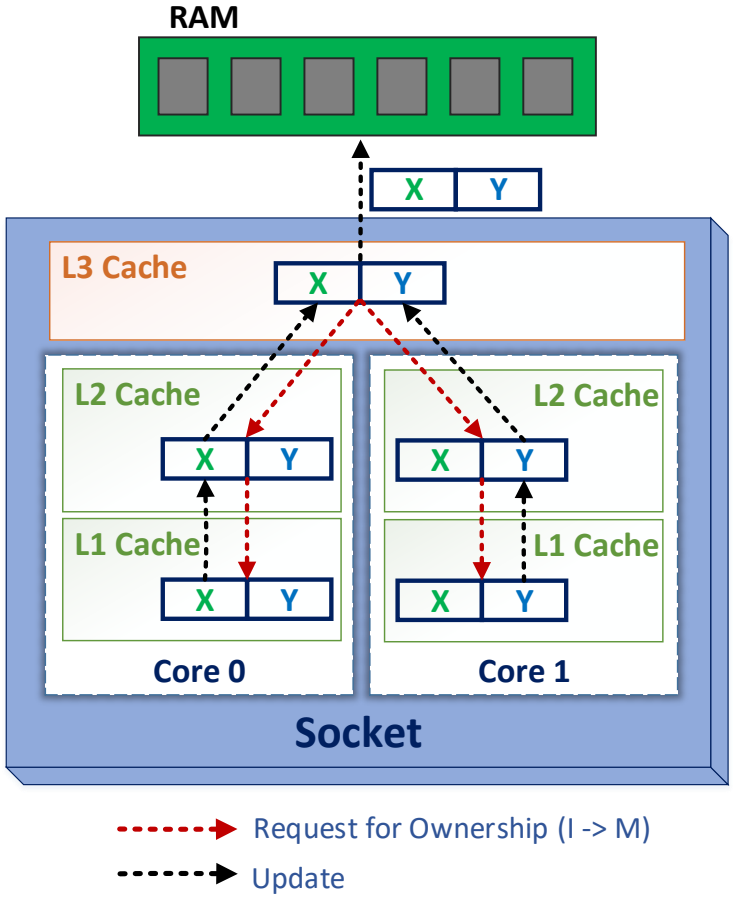
```
public class FalseSharing {
    public int X;
    public int Y;
}

FalseSharing sharedInstance = new FalseSharing();
```

Thread 1

```
void incrementX () {
    sharedInstance.X ++
}
```

Thread 2

```
void incrementY () {
    sharedInstance.Y ++
}
```



False Sharing

Guidelines

- independent values sits on the **same cache line**
- different cores **concurrently access** that line
- there is at **least one writer** thread
- **high frequency** of writing/reading

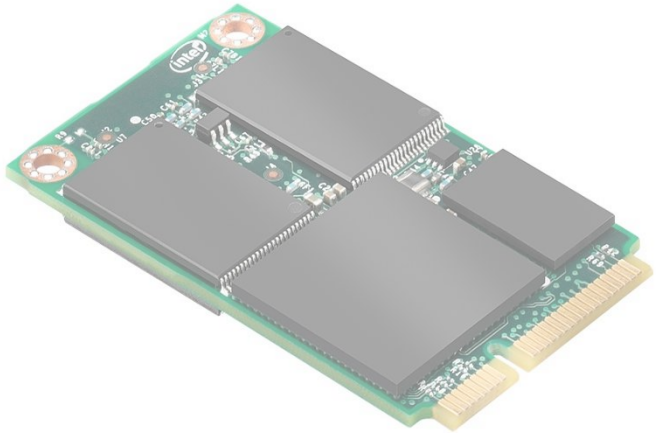
Solid State Drive

TRIM

ON | OFF

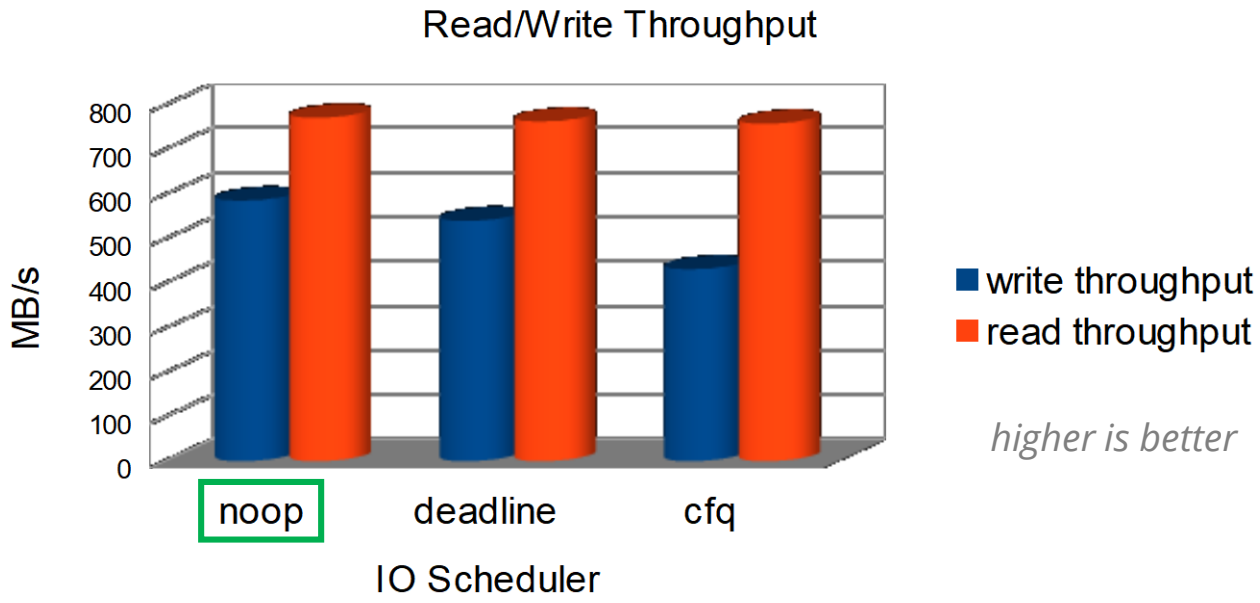
I/O Scheduler

NOOP | Deadline | CFQ



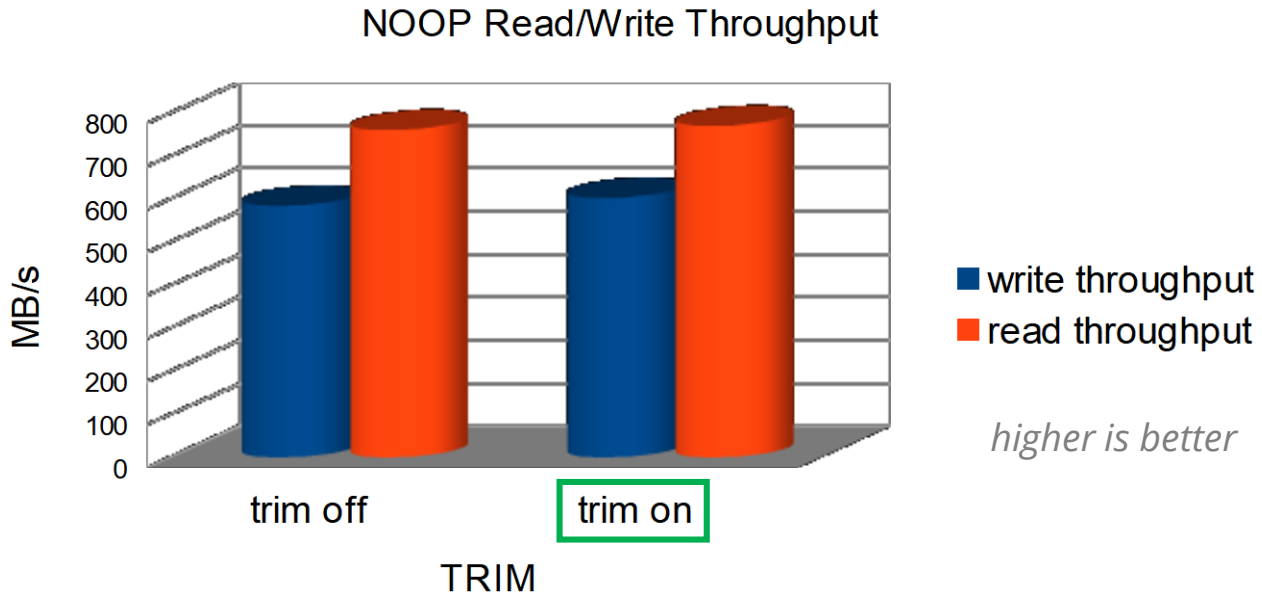
Solid State Drive

Test scenario: sequentially writing/reading 32GB in chunks of 512 KB on SSD

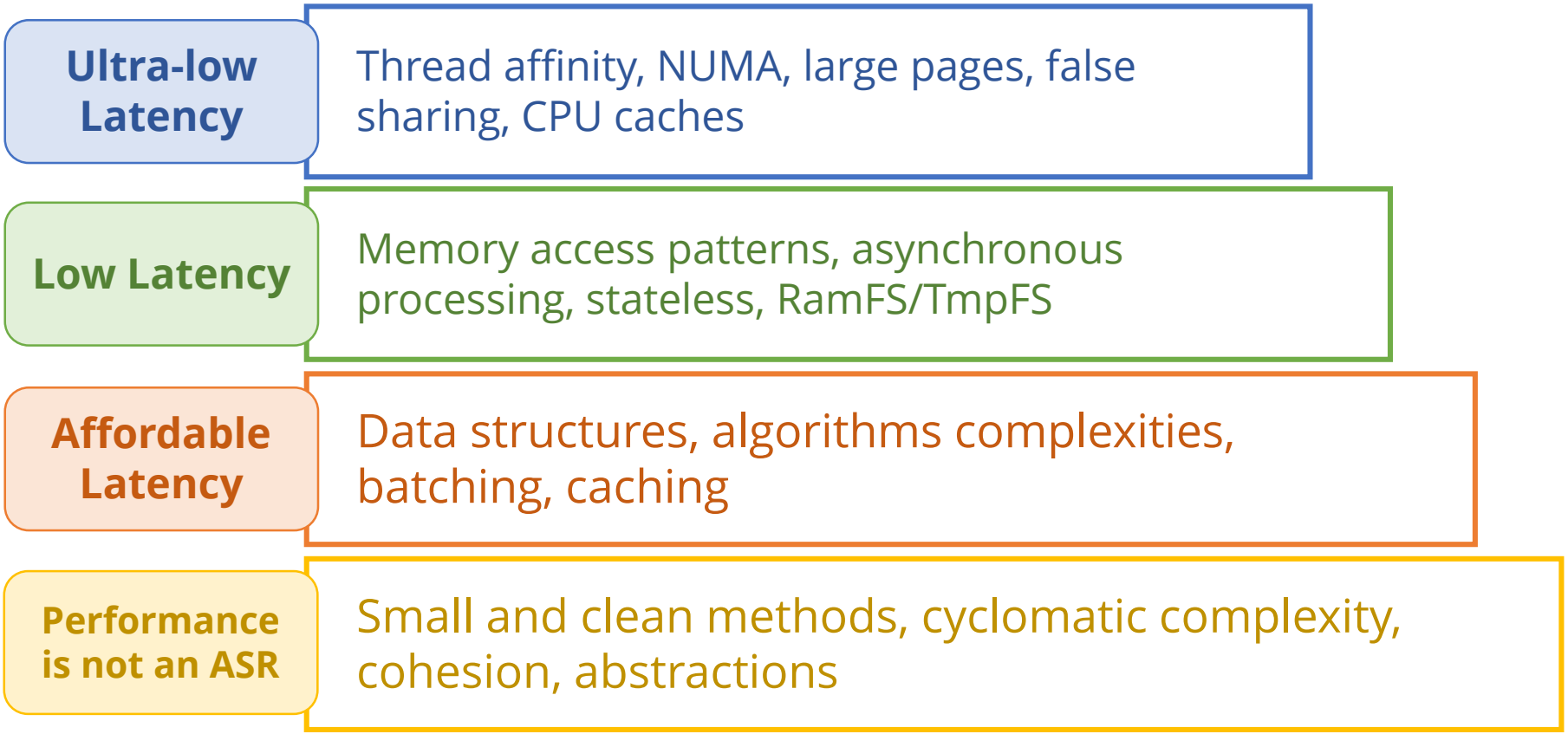


Solid State Drive

Test scenario: sequentially writing/reading 32GB in chunks of 512 KB on SSD



My Latency Hierarchical Model



NB: Model is not exclusive and might be subject of changes

Performance is simple, you just have to be aware of everything!

Ionuţ Baloşin

Thank You

I T.A.K.E. Unconference | 2018

Ionuţ Baloşin

Software Architect

@ionutbalosin

Further References

Articles by **Ulrich Drepper**

- ✓ What every programmer should know about memory
- ✓ CPU caches
- ✓ Virtual memory
- ✓ NUMA systems
- ✓ What programmers can do - cache optimization
- ✓ What programmers can do - multi-threaded optimizations
- ✓ Memory performance tools

Further References

- ✓ **Performance Methodology Mindmap - Kirk Perpendine and Alexey Shipilev**
  - https://shipilev.net/talks/devoxx-Nov2012-perfMethodology-mindmap.pdf
- ✓ **Cpu Caches and Why You Care - Scott Meyers**
- ✓ **CPU caches - Ulrich Drepper**
- ✓ **Async or Bust!? - Todd Montgomery**
- ✓ http://mechanical-sympathy.blogspot
- ✓ **An Introduction to Lock-Free Programming**
  - http://preshing.com/20120612/an-introduction-to-lock-free-programming
- ✓ **Intel's 'cmpxchg' instruction**
  - http://heather.cs.ucdavis.edu/~matloff/50/PLN/lock.pdf
- ✓ http://docs.oracle.com/javase/7/docs/technotes/guides/vm/performance-enhancements-7.html
- ✓ http://www.thegeekstuff.com/2008/11/overview-of-ramfs-and-tmpfs-on-linux

Thank You

I T.A.K.E. Unconference | 2018

Ionuţ Baloşin

Software Architect

@ionutbalosin